

Análisis técnico de arquitecturas y herramientas de procesamiento de datos en streaming en el ecosistema de Big Data

El panorama del procesamiento de datos ha experimentado una transformación radical desde la concepción original del ecosistema Hadoop. En sus inicios, el procesamiento por lotes (*batch processing*) era la norma, aceptando latencias de horas o días para obtener resultados analíticos. Sin embargo, la madurez de la economía digital y la explosión de la Internet de las Cosas (IoT) han desplazado el centro de gravedad hacia la inmediatez. En este contexto, el procesamiento de datos en streaming no es simplemente una mejora de rendimiento, sino una necesidad arquitectónica para organizaciones que requieren reaccionar a eventos en el momento en que ocurren.¹ El presente informe técnico desglosa las principales herramientas para el tratamiento de datos en streaming —Apache Flink, Apache Spark, Apache Kafka y Apache Samza— analizando sus arquitecturas internas, sus mecanismos de garantía de procesamiento y los escenarios específicos donde cada una ofrece una ventaja competitiva diferencial.

Evolución del paradigma: Del procesamiento por lotes al flujo continuo

La transición de los sistemas tradicionales a los de tiempo real ha estado marcada por la búsqueda de un equilibrio entre tres variables críticas: latencia, rendimiento (*throughput*) y consistencia. Los primeros sistemas de Big Data, como MapReduce, priorizaban el rendimiento a costa de una latencia extremadamente alta debido a la persistencia constante en disco de los resultados intermedios.¹ Posteriormente, surgieron sistemas de streaming de primera generación como Apache Storm, que ofrecían baja latencia pero carecían de una gestión de estado robusta y garantías de procesamiento consistentes, limitándose a menudo a una semántica de "al menos una vez".³

La generación actual, liderada por Flink y Spark Structured Streaming, ha resuelto gran parte de estos dilemas mediante la introducción de conceptos avanzados como el tiempo del evento (*event-time*), las marcas de agua (*watermarks*) y el procesamiento de estado distribuido con garantías de "exactamente una vez" (*exactly-once semantics*).⁴ Esta evolución permite que los flujos de datos no acotados (*unbounded*) se traten con el mismo rigor analítico que los conjuntos de datos finitos o acotados (*bounded*).⁶

Apache Flink: El motor de streaming nativo y de

estado distribuido

Apache Flink representa la vanguardia del procesamiento de flujos de datos de baja latencia y alta consistencia. Diseñado desde su origen como un motor de streaming puro, Flink trata el procesamiento por lotes simplemente como un caso especial de flujos de datos finitos, lo que le permite ofrecer una arquitectura unificada sin los compromisos de latencia asociados a los sistemas basados en micro-lotes.⁴

Arquitectura interna y modelo de ejecución

A diferencia de otros marcos de trabajo, Flink opera mediante un modelo de ejecución basado en un grafo acíclico dirigido (DAG) de operadores que procesan eventos uno a uno en una tubería continua (*pipelined execution*). Esta arquitectura permite que los datos fluyan a través de los nodos de computación sin necesidad de esperar a que se acumulen en lotes, lo que reduce la latencia de extremo a extremo a niveles de milisegundos.⁷

El clúster de Flink se organiza típicamente en dos componentes principales: el JobManager y los TaskManagers. El JobManager actúa como el orquestador central, responsable de la programación de tareas, la gestión de puntos de control (*checkpoints*) y la recuperación ante fallos. Por su parte, los TaskManagers son los trabajadores que ejecutan las tareas individuales y gestionan los buffers de datos y el estado local de los operadores.⁶ En entornos modernos, Flink se integra de forma nativa con Kubernetes y YARN, permitiendo una gestión de recursos dinámica y elástica que se ajusta a las fluctuaciones del tráfico de entrada.¹⁰

Gestión de estado y tolerancia a fallos

La característica más distintiva de Flink es su manejo del estado de la aplicación. Para operaciones que requieren memoria de eventos pasados, como agregaciones en ventanas de tiempo o uniones (*joins*) de flujos, Flink mantiene el estado localmente en cada TaskManager utilizando *backends* de estado especializados. El más común para cargas de producción es RocksDB, que permite gestionar estados de varios terabytes al desbordar datos de la memoria al disco local.⁴

La tolerancia a fallos se implementa mediante un algoritmo de instantáneas distribuidas basado en una variante del protocolo Chandy-Lamport. Este mecanismo inyecta barreras especiales en el flujo de datos; cuando un operador recibe estas barreras, realiza una copia asíncrona de su estado actual en un almacenamiento duradero (como S3 o HDFS).⁵ En caso de fallo, el sistema reinicia desde el último punto de control consistente, garantizando que cada registro afecte al resultado final exactamente una vez.⁴

Casos de aplicación específicos de Apache Flink

Apache Flink es la herramienta de elección en escenarios donde la precisión absoluta y la latencia sub-segundo son críticas. Su capacidad para manejar el tiempo del evento y datos desordenados mediante *watermarks* lo hace superior para análisis complejos.⁴

Escenario de Aplicación	Requerimiento Técnico Crítico	Beneficio de Flink
Detección de fraude financiero	Latencia < 100ms y consistencia exacta.	Procesa transacciones en tiempo real con estado histórico para detectar patrones sospechosos inmediatamente. ¹³
Sistemas de recomendación en tiempo real	Actualización de modelos basada en el comportamiento actual del usuario.	Permite unir flujos de clics con el historial del usuario para personalizar la experiencia durante la sesión. ¹⁴
Monitorización de redes de telecomunicaciones	Procesamiento de billones de eventos con baja latencia.	Bouygues Telecom utiliza Flink para monitorizar la calidad del servicio con una latencia inferior a 200ms en clústeres optimizados. ¹⁴
Juegos en línea	Análisis de telemetría y equilibrio de carga.	King (Candy Crush) gestiona 30 mil millones de eventos diarios para ajustar la dificultad y detectar anomalías en tiempo real. ¹⁴

Apache Spark Structured Streaming: Unificación y simplicidad a escala

Apache Spark es posiblemente el marco de trabajo de Big Data más adoptado en la industria debido a su versatilidad y su amplio ecosistema. Structured Streaming es el motor de procesamiento de flujos construido sobre Spark SQL, diseñado para simplificar la creación de tuberías de datos en tiempo real permitiendo a los usuarios expresar sus consultas de streaming de la misma manera que lo harían para el procesamiento por lotes.¹⁵

Modelo de micro-lotes frente a procesamiento continuo

Spark Structured Streaming opera fundamentalmente mediante un modelo de micro-lotes (*micro-batching*). En este enfoque, el sistema recibe el flujo de datos y lo divide en pequeños lotes discretos (por ejemplo, cada 500ms o 1s), tratándolos como trabajos de Spark altamente optimizados. Este modelo hereda la eficiencia del optimizador Catalyst de Spark y permite un

alto rendimiento al amortizar los costes fijos de procesamiento entre múltiples registros.⁷

Sin embargo, el modelo de micro-lotes introduce una latencia mínima infranqueable asociada a la programación del lote y la persistencia de los logs de progreso. Para abordar casos de uso que requieren latencias aún menores, Spark introdujo el "Continuous Processing Mode", que permite procesar registros uno a uno con latencias de un solo dígito de milisegundo, aunque con restricciones en las operaciones soportadas y garantías de "al menos una vez".¹⁹

Recientemente, el desarrollo del "Real-Time Mode" (RTM) busca cerrar la brecha con Flink al reducir la sobrecarga de los micro-lotes tradicionales.¹⁷

Integración con el ecosistema analítico

La mayor fortaleza de Spark reside en su integración perfecta con otras bibliotecas del ecosistema:

- **MLlib:** Permite entrenar modelos de aprendizaje automático sobre datos históricos y aplicarlos inmediatamente a flujos de streaming para tareas de clasificación o regresión en tiempo real.²³
- **GraphX:** Facilita el procesamiento de grafos sobre datos que fluyen, lo cual es esencial para el análisis de redes sociales o la detección de comunidades dinámicas.²⁵
- **Delta Lake / Lakehouse:** Structured Streaming es el motor principal para alimentar arquitecturas de Data Lakehouse, permitiendo la ingesta continua y la compactación de archivos de forma transparente.²⁷

Casos de aplicación específicos de Apache Spark

Spark es ideal para organizaciones que ya cuentan con infraestructura de Spark y cuyos casos de uso toleran latencias en el rango de los segundos pero requieren una integración analítica profunda.⁸

Escenario de Aplicación	Requerimiento Técnico Crítico	Beneficio de Spark
Ingesta de datos en Data Lakes (ETL)	Rendimiento masivo y limpieza de esquemas.	Facilita la transformación y carga de datos de Kafka a S3/Delta Lake con esquemas definidos y evolución de los mismos. ²⁷
Análisis de logs y ciberseguridad	Correlación de eventos históricos con el flujo actual.	Permite unir flujos de logs de red con bases de datos de amenazas históricas para identificar intrusiones

		mediante ML. ²⁵
Business Intelligence en tiempo real	Dashboards que se actualizan cada pocos segundos.	Integración directa con Spark SQL para servir consultas analíticas sobre flujos de ventas o clics. ¹⁶
Monitorización de salud del sistema	Detección de tendencias y anomalías en series temporales.	Uso de ventanas de tiempo deslizantes para calcular medias móviles y disparar alertas operativas. ¹⁸

Apache Kafka Streams: El poder del streaming embebido en aplicaciones

Apache Kafka es reconocido mundialmente como la columna vertebral de la mensajería en tiempo real. Kafka Streams, introducido en la versión 0.10, es una biblioteca cliente (no un clúster separado) que permite realizar procesamientos de flujos directamente sobre temas de Kafka.³⁰

Arquitectura de biblioteca y despliegue ligero

A diferencia de Flink o Spark, que requieren un clúster dedicado para la computación, Kafka Streams es una biblioteca que se importa en una aplicación Java o Scala convencional. Esto significa que la lógica de procesamiento vive dentro del proceso de la aplicación, lo que simplifica enormemente el despliegue y la operación. Las aplicaciones de Kafka Streams pueden ejecutarse en contenedores estándar, ser orquestadas por Kubernetes o incluso ejecutarse en máquinas virtuales aisladas.³⁰

La arquitectura se basa en una topología de procesadores donde los nodos representan operaciones (como filtrar, mapear o agregar) y los bordes representan flujos de datos. La escalabilidad se logra de forma nativa mediante el particionamiento de Kafka: el número de instancias de la aplicación puede escalar horizontalmente hasta igualar el número de particiones del tema de entrada.³⁰

La dualidad Stream-Table y las transacciones

Kafka Streams introdujo un concepto fundamental: la dualidad entre flujos (*KStream*) y tablas (*KTable*). Un flujo es una secuencia infinita de eventos, mientras que una tabla representa el estado actual resultante de esos eventos. Esta abstracción facilita enormemente la construcción de aplicaciones de estado, como uniones entre flujos de transacciones y tablas de perfiles de usuario.³⁰

Para garantizar la consistencia, Kafka Streams utiliza las capacidades transaccionales nativas

de Kafka. En un ciclo de "lectura-procesamiento-escritura", los desplazamientos (*offsets*) de entrada, el estado local y los registros de salida se confirman de forma atómica. Si ocurre un fallo, la transacción se aborta y el sistema vuelve al último estado consistente conocido, proporcionando semántica de "exactamente una vez" sin la necesidad de un coordinador externo pesado.³⁶

Casos de aplicación específicos de Apache Kafka Streams

Kafka Streams es la herramienta predominante para arquitecturas de microservicios y sistemas donde el procesamiento debe estar lo más cerca posible del flujo de datos.²⁷

Escenario de Aplicación	Requerimiento Técnico Crítico	Beneficio de Kafka Streams
Microservicios de banca	Gestión de saldos y transferencias con alta disponibilidad.	Garantiza que cada operación financiera se procese exactamente una vez mediante transacciones de Kafka. ³⁶
Enriquecimiento de flujos en tiempo real	Unión de eventos de entrada con metadatos de referencia.	Uso de <i>GlobalKTable</i> para mantener datos de referencia en memoria en todas las instancias de procesamiento. ³⁰
Seguimiento de inventario minorista	Actualizaciones rápidas del stock basadas en ventas.	Procesa eventos de compra individuales y actualiza el estado del inventario con latencias de milisegundos. ³⁸
Análisis de sensores IoT en el borde	Procesamiento ligero sin necesidad de clústeres pesados.	Al ser una biblioteca, puede desplegarse en dispositivos locales o puertas de enlace IoT con recursos limitados. ³⁴

Apache Samza: Arquitectura basada en logs y gestión de estados masivos

Desarrollado originalmente en LinkedIn para alimentar sus sistemas de notificaciones y analíticas, Apache Samza ofrece una visión del procesamiento de flujos centrada en la

robustez y la afinidad de los datos con el almacenamiento.⁴⁰

Integración con Kafka y YARN

La arquitectura de Samza está estrechamente ligada a Apache Kafka (como capa de transporte de mensajes) y Apache Hadoop YARN (como capa de gestión de recursos). Samza descompone las aplicaciones en tareas de procesamiento que se ejecutan dentro de contenedores gestionados por YARN. Cada tarea consume datos de una partición específica de Kafka, lo que permite un aislamiento de recursos granular y una escalabilidad lineal.⁴²

A diferencia de Flink, que utiliza un enfoque de instantáneas globales, Samza se basa en el concepto de "dar la vuelta a la base de datos" (*turning the database inside out*), donde el log de Kafka es la fuente de la verdad y el estado local de la aplicación es una vista materializada de ese log.⁴⁴

Gestión de estado y afinidad de host

Samza es particularmente fuerte en la gestión de estados masivos de varios terabytes. Utiliza RocksDB para el almacenamiento de estado local y garantiza la durabilidad replicando cada cambio en el estado hacia un tema de registro de cambios (*changelog topic*) en Kafka.¹³ Un aspecto diferenciador es su "afinidad de host": cuando una tarea falla, YARN intenta reiniciarla en el mismo nodo físico donde residía anteriormente. Esto permite que la tarea reutilice el estado local persistido en disco, evitando el tiempo de reconstrucción a través de la red, lo cual es crítico para aplicaciones con estados extremadamente grandes.⁴¹

Casos de aplicación específicos de Apache Samza

Samza se aplica preferentemente en infraestructuras maduras de Hadoop y en escenarios de red social con necesidades de estado muy elevadas.⁴⁰

Escenario de Aplicación	Requerimiento Técnico Crítico	Beneficio de Samza
Monitorización de infraestructura crítica	Análisis de flujos de métricas de servidores a escala global.	Slack utiliza Samza para sus pipelines de datos de monitorización debido a su estabilidad operativa. ⁴⁶
Optimización de comunicaciones de usuario	Decidir en tiempo real si enviar un email o notificación.	LinkedIn emplea Samza para su "Air Traffic Controller", gestionando estados complejos sobre millones de usuarios. ⁴⁶

Prevención de fraude a escala web	Análisis de patrones de navegación en sitios de alto tráfico.	eBay utiliza Samza para procesar eventos de usuario y detectar actividades maliciosas con baja latencia. ⁴⁶
Re-procesamiento de datos históricos	Capacidad de "rebobinar" y recalcular resultados.	Al basarse en el log de Kafka, permite reiniciar trabajos desde el principio para corregir errores o aplicar nuevas lógicas analíticas. ⁴³

Comparativa técnica exhaustiva: Rendimiento y capacidades

La elección entre estas herramientas requiere un análisis comparativo de sus capacidades bajo diferentes dimensiones operativas.

Métricas de rendimiento y latencia

El rendimiento de un sistema de streaming se define por su latencia (el tiempo que tarda un evento individual en ser procesado) y su *throughput* (el volumen de datos procesados por unidad de tiempo). La siguiente tabla resume el comportamiento típico observado en entornos de producción.

Herramienta	Latencia Típica (p99)	Modelo de Procesamiento	Garantía de Entrega
Apache Flink	50ms – 500ms	Streaming Nativo	Exactamente una vez (EOS) ⁸
Spark Structured Streaming	500ms – 5s	Micro-lotes	Exactamente una vez (EOS) ⁸
Kafka Streams	100ms – 1000ms	Streaming Nativo (App Polling)	Exactamente una vez (Transaccional) ⁸
Apache Samza	100ms – 1000ms	Streaming Nativo (Basado en Tareas)	Al menos una vez / EOS ⁴⁰

Desde una perspectiva matemática, la latencia en un sistema de micro-lotes como Spark puede modelarse como:

$$L \approx \frac{1}{2}T_{batch} + T_{exec} + T_{overhead}$$

Donde T_{batch} es el intervalo del disparador y T_{exec} es el tiempo de ejecución real del lote. En contraste, en un sistema nativo como Flink, la latencia es fundamentalmente:

$$L \approx T_{proc} + T_{network}$$

Lo que explica por qué Flink puede alcanzar latencias inferiores de forma consistente en aplicaciones de alta frecuencia.⁷

Capacidad de gestión de estado

La gestión de estado es el mayor desafío técnico en el procesamiento distribuido. La capacidad de un motor para manejar estados que no caben en RAM determina su utilidad para análisis de larga duración.

Dimensión	Apache Flink	Spark Structured Streaming	Kafka Streams	Apache Samza
Backend de Estado	RocksDB / HashMap	Memoria / HDFS / RocksDB	RocksDB / In-memory	RocksDB
Escalado del Estado	Terabytes por clúster	Limitado por RAM / Delta Lake	Decenas de GB por nodo	Terabytes (Afinidad de Host)
Recuperación	Instantáneas asíncronas	Re-ejecución de lotes	Replay del Changelog	Replay / Reutilización local
Consulta	Queryable State	A través de	Interactive	No nativo

Externa	(Beta)	tablas externas	Queries	
---------	--------	-----------------	---------	--

Flink y Samza son superiores cuando se trata de gestionar estados masivos que exceden la memoria física del clúster.⁵ Kafka Streams es excelente para estados moderados donde la simplicidad de la recuperación a través de Kafka es una ventaja.⁵ Spark, por su parte, tiende a ser más eficiente cuando el estado puede ser procesado en memoria o persistido en tablas de un Data Lakehouse como Delta Lake.⁸

Criterios de aplicación y selección de herramientas

Para determinar qué herramienta aplicar en un proyecto de Big Data, se debe realizar un triaje basado en los requisitos funcionales y operativos.

Cuándo aplicar Apache Flink

Flink debe ser la elección predeterminada cuando el caso de uso requiere una latencia sub-segundo combinada con una lógica de tiempo compleja. Es especialmente valioso en situaciones donde los datos pueden llegar significativamente tarde o fuera de orden, ya que su motor de *watermarks* permite una gestión precisa del tiempo del evento.⁴ Organizaciones con equipos de ingeniería de plataformas capaces de gestionar clústeres dedicados encontrarán en Flink el motor más potente para el procesamiento de eventos complejos (CEP).¹³

Cuándo aplicar Apache Spark Structured Streaming

Spark es la opción más lógica si la organización ya utiliza Spark para el procesamiento por lotes o el aprendizaje automático. La capacidad de reutilizar código y lógica entre batch y streaming reduce drásticamente el tiempo de desarrollo. Es la herramienta ideal para tuberías de ingesta masiva (ETL) y para casos donde el análisis requiere unir flujos de datos con grandes volúmenes de datos históricos almacenados en disco.⁷

Cuándo aplicar Apache Kafka Streams

Se debe elegir Kafka Streams cuando el objetivo es construir aplicaciones ágiles y microservicios orientados a eventos. Si el procesamiento es ligero o moderado y no se desea la sobrecarga operativa de gestionar un clúster de computación separado (como YARN o un clúster independiente de Flink), Kafka Streams ofrece la mejor relación entre potencia y simplicidad.²⁷ Es particularmente fuerte en arquitecturas "Kafka-nativas" donde tanto la entrada como la salida son temas de Kafka.

Cuándo aplicar Apache Samza

Samza es la herramienta recomendada para empresas con una inversión masiva en el ecosistema Hadoop y que necesitan procesar flujos de datos con estados locales gigantescos.

Su diseño centrado en la afinidad de host y su integración profunda con el modelo de particionamiento de Kafka lo hacen extremadamente estable para cargas de trabajo predecibles y de gran escala que requieren una recuperación de fallos optimizada para el almacenamiento local.⁴⁰

El impacto de las garantías de procesamiento: At-Least-Once vs Exactly-Once

La fiabilidad de los resultados analíticos depende de las garantías de procesamiento que ofrece la herramienta.

1. **At-Most-Once:** Los datos pueden perderse pero nunca duplicarse. Se utiliza raramente en aplicaciones financieras, pero puede ser aceptable para métricas de monitorización no críticas donde la velocidad es la única prioridad.⁵⁰
2. **At-Least-Once:** Los datos nunca se pierden, pero pueden procesarse varias veces en caso de fallo. Es la garantía predeterminada en muchos sistemas y requiere que las aplicaciones finales sean idempotentes (que procesar el mismo dato dos veces produzca el mismo resultado).⁵⁰
3. **Exactly-Once:** Es el "estándar de oro". El sistema garantiza que cada evento afecte al estado y a los resultados finales exactamente una vez, incluso ante fallos de red o de nodos. Flink logra esto mediante puntos de control coordinados, Spark mediante el linaje de micro-lotes, y Kafka Streams mediante transacciones atómicas.⁷

La elección de una garantía más fuerte (Exactly-Once) suele conllevar una penalización en la latencia y un aumento en la complejidad de configuración, pero es esencial para aplicaciones de facturación, gestión de inventario y cumplimiento regulatorio.³⁸

Consideraciones operativas y de despliegue en la nube

En el contexto actual de Big Data, la facilidad de operación es tan importante como el rendimiento técnico.

- **Despliegue Cloud-Native:** Flink y Spark han evolucionado hacia modelos nativos en Kubernetes, lo que permite el auto-escalado dinámico. Managed Services como Amazon Managed Service for Apache Flink o Databricks eliminan la carga de gestionar el plano de control.⁶
- **Monitoreo y Observabilidad:** Kafka Streams, al ser una aplicación Java, puede monitorearse con herramientas estándar como Prometheus y Grafana. Flink ofrece un panel de control detallado para visualizar el grafo de ejecución y el progreso de los puntos de control en tiempo real.³⁰
- **Coste Operativo:** Kafka Streams suele tener el TCO (Coste Total de Propiedad) más bajo para aplicaciones pequeñas y medianas debido a su falta de infraestructura de

clúster. Spark y Flink requieren una inversión mayor en infraestructura y personal especializado para el ajuste de rendimiento de los clústeres.⁸

Conclusión sobre la arquitectura de streaming ideal

El tratamiento de datos en streaming en el contexto de Big Data ha dejado de ser una disciplina de nicho para convertirse en una competencia central de la ingeniería de datos moderna. No existe una herramienta única que sea superior en todos los aspectos; la selección debe basarse en un compromiso técnico consciente.

Si el objetivo es la latencia más baja posible con una semántica de tiempo del evento rigurosa y procesamiento de estado a gran escala, **Apache Flink** se posiciona como el líder indiscutible. Para organizaciones que buscan una plataforma unificada para análisis de datos, aprendizaje automático e ingesta en un Data Lakehouse, **Apache Spark Structured Streaming** ofrece la mejor integración y facilidad de uso. En el mundo de los microservicios y el desarrollo de aplicaciones ligeras y escalables, **Apache Kafka Streams** proporciona una agilidad y simplicidad operativa sin igual. Finalmente, para escenarios de escala extrema con estados locales masivos integrados en entornos de Hadoop, **Apache Samza** sigue ofreciendo una robustez y eficiencia de recuperación difícil de igualar.

La tendencia futura apunta hacia una convergencia donde estas herramientas adoptarán capacidades de las otras (como Flink mejorando su soporte SQL o Spark reduciendo su latencia de micro-lotes), pero sus filosofías arquitectónicas subyacentes seguirán dictando su idoneidad para casos de uso específicos. La clave del éxito en una estrategia de streaming reside en alinear estas capacidades técnicas con los SLAs de negocio y la madurez operativa de la organización.

Obras citadas

1. Kafka, Samza and the Unix Philosophy of Distributed Data, fecha de acceso: mayo 5, 2026,
<https://api.repository.cam.ac.uk/server/api/core/bitstreams/bfa1a4b9-3d3d-4cae-8e26-f26fea29f9f6/content>
2. Hadoop, Storm, Samza, Spark, and Flink: Big Data Frameworks, fecha de acceso: mayo 5, 2026,
<https://www.digitalocean.com/community/tutorials/hadoop-storm-samza-spark-and-flink-big-data-frameworks-compared>
3. What is/are the main difference(s) between Flink and Storm?, fecha de acceso: mayo 5, 2026,
<https://stackoverflow.com/questions/30699119/what-is-are-the-main-differences-between-flink-and-storm>
4. Flink State Management: A Journey from Core Primitives to Next, fecha de acceso: mayo 5, 2026,
https://www.alibabacloud.com/blog/flink-state-management-a-journey-from-core-primitives-to-next-generation-incremental-computation_602503

5. Kafka Streams vs Apache Flink: When to Use What - Conduktor, fecha de acceso: mayo 5, 2026, <https://www.conduktor.io/glossary/kafka-streams-vs-apache-flink>
6. Apache Flink? — Architecture, fecha de acceso: mayo 5, 2026, <https://flink.apache.org/what-is-flink/flink-architecture/>
7. Spark Streaming vs Apache Flink®: Modern Stream Processing ..., fecha de acceso: mayo 5, 2026, <https://www.confluent.io/compare/spark-streaming-vs-flink/>
8. Flink vs Spark Streaming vs Kafka Streams: Stream Engines (2026), fecha de acceso: mayo 5, 2026, <https://iotdigitaltwinplm.com/flink-vs-spark-streaming-vs-kafka-streams-comparison-2026/>
9. Principles and Practices of Flink on YARN and Kubernetes, fecha de acceso: mayo 5, 2026, https://www.alibabacloud.com/blog/principles-and-practices-of-flink-on-yarn-and-kubernetes-flink-advanced-tutorials_596625
10. How to natively deploy Flink on Kubernetes with High-Availability (HA), fecha de acceso: mayo 5, 2026, <https://flink.apache.org/2021/02/10/how-to-natively-deploy-flink-on-kubernetes-with-high-availability-ha/>
11. State Management in Apache FlinkR, fecha de acceso: mayo 5, 2026, <https://id2221kth.github.io/papers/2017%20-%20State%20Management%20in%20Apache%20Flink.pdf>
12. Deep dive into the Amazon Managed Service for Apache Flink, fecha de acceso: mayo 5, 2026, <https://aws.amazon.com/blogs/big-data/deep-dive-into-the-amazon-managed-service-for-apache-flink-application-lifecycle-part-1/>
13. Top 5 Stream Processing Engines Compared (2026) - Upstaff.com, fecha de acceso: mayo 5, 2026, <https://upstaff.com/blog/engineering/top-5-stream-processing-engines-a-comprehensive-comparison/>
14. Real life case studies of Apache Flink - DataFlair, fecha de acceso: mayo 5, 2026, <https://data-flair.training/blogs/apache-flink-use-cases/>
15. Structured Streaming concepts | Databricks on AWS, fecha de acceso: mayo 5, 2026, <https://docs.databricks.com/aws/en/structured-streaming/concepts>
16. Spark Streaming: Real-Time Data Magic - Euron Daily, fecha de acceso: mayo 5, 2026, <https://euron.one/community/posts/db135257-a6c8-4197-b238-4a0dfe32a1d2>
17. The Architecture of Apache Spark Real-Time Mode | Databricks Blog, fecha de acceso: mayo 5, 2026, <https://www.databricks.com/blog/breaking-microbatch-barrier-architecture-apache-spark-real-time-mode>
18. Why Spark Structured Streaming Could Be The Best Choice - Netguru, fecha de acceso: mayo 5, 2026, <https://www.netguru.com/blog/spark-streaming>
19. Performance Benchmarking of Continuous Processing and Micro, fecha de acceso: mayo 5, 2026, <https://ceur-ws.org/Vol-3896/paper5.pdf>
20. Structured Streaming Programming Guide - Spark 4.1.1, fecha de acceso: mayo

- 5, 2026, <https://spark.apache.org/docs/latest/streaming/performance-tips.html>
21. Structured Streaming Programming Guide - Spark 4.1.1 ..., fecha de acceso: mayo 5, 2026, <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
 22. Introducing Real-Time Mode in Apache Spark™ Structured Streaming, fecha de acceso: mayo 5, 2026, <https://www.databricks.com/blog/introducing-real-time-mode-apache-sparktm-structured-streaming>
 23. Machine Learning Library (MLlib) Guide - Apache Spark, fecha de acceso: mayo 5, 2026, <https://spark.apache.org/docs/latest/ml-guide.html>
 24. Machine Learning with Spark Streaming - Clairvoyant Blog, fecha de acceso: mayo 5, 2026, <https://blog.clairvoyantsoft.com/machine-learning-with-spark-streaming-281b2d1e4fd5>
 25. Streaming Data Processing with Apache Kafka and Spark, fecha de acceso: mayo 5, 2026, <https://admin.mantechpublications.com/index.php/RTiCSaST/issue/download/5832/6419>
 26. Spark Streaming and GraphX - Amir H. Payberah, fecha de acceso: mayo 5, 2026, https://payberah.github.io/files/download/slides/eit_spark_streaming_graphx.pdf
 27. Stream Processing Tools Compared: Flink, Kafka Streams, Spark, fecha de acceso: mayo 5, 2026, <https://streamkap.com/resources-and-guides/stream-processing-tools-compared>
 28. Top Trends for Data Streaming with Apache Kafka and Flink in 2025, fecha de acceso: mayo 5, 2026, <https://www.kai-waehner.de/blog/2024/12/02/top-trends-for-data-streaming-with-apache-kafka-and-flink-in-2025/>
 29. Optimizing Spark Structured Streaming on Databricks | SoftServe, fecha de acceso: mayo 5, 2026, <https://www.softserveinc.com/en-us/blog/optimizing-spark-structured-streaming>
 30. What is Kafka Streams? Concepts, Examples & Best Practices, fecha de acceso: mayo 5, 2026, <https://www.automq.com/blog/kafka-streams-guide-concepts-examples-best-practices>
 31. Introduction | Apache Kafka, fecha de acceso: mayo 5, 2026, <https://kafka.apache.org/documentation/>
 32. Flink vs Kafka Streams: A Complete Comparison - Confluent, fecha de acceso: mayo 5, 2026, <https://www.confluent.io/blog/apache-flink-apache-kafka-streams-comparison-guide-for-users/>
 33. Kafka Streams—a deep dive - Redpanda, fecha de acceso: mayo 5, 2026, <https://www.redpanda.com/guides/kafka-architecture-kafka-streams>
 34. Kafka Streams vs. Apache Flink | OpenLogic, fecha de acceso: mayo 5, 2026, <https://www.openlogic.com/blog/apache-flink-vs-kafka-streams>

35. How Kafka Streams work and their key benefits - DoubleCloud, fecha de acceso: mayo 5, 2026, <https://double.cloud/blog/posts/2024/05/kafka-streams/>
36. Kafka Transactional Support: How It Enables Exactly-Once Semantics, fecha de acceso: mayo 5, 2026, <https://developer.confluent.io/courses/architecture/transactions/>
37. Kafka Transactions Deep Dive - Conduktor, fecha de acceso: mayo 5, 2026, <https://www.conduktor.io/glossary/kafka-transactions-deep-dive>
38. Exactly-Once Semantics in Kafka - Conduktor, fecha de acceso: mayo 5, 2026, <https://www.conduktor.io/glossary/exactly-once-semantics-in-kafka>
39. Kafka Streams vs Apache Flink: A Pragmatic Comparison ... - Medium, fecha de acceso: mayo 5, 2026, <https://medium.com/@souquieres.adam/kafka-streams-vs-apache-flink-a-pragmatic-comparison-for-stream-processing-and-why-you-should-66fc0b641b26>
40. What is Apache Samza? - Dremio, fecha de acceso: mayo 5, 2026, <https://www.dremio.com/wiki/apache-samza/>
41. The Past and Present of Stream Processing (Part 6) — Apache Samza, fecha de acceso: mayo 5, 2026, <https://taogang.medium.com/the-evolution-of-stream-processing-part-6-apache-samza-the-middle-path-born-at-the-wrong-time-d75f0b91e16c>
42. Architecture page - Apache Samza, fecha de acceso: mayo 5, 2026, <https://samza.apache.org/learn/documentation/latest/architecture/architecture-overview.html>
43. Samza - Architecture - Apache Samza, fecha de acceso: mayo 5, 2026, <https://samza.apache.org/learn/documentation/latest/introduction/architecture.html>
44. Turning the database inside-out with Apache Samza | Confluent, fecha de acceso: mayo 5, 2026, <https://www.confluent.io/blog/turning-the-database-inside-out-with-apache-samza/>
45. Turning the database inside out with Apache Samza, fecha de acceso: mayo 5, 2026, <https://martin.kleppmann.com/2014/09/18/turning-database-inside-out-at-strange-loop.html>
46. Case Studies - Apache Samza, fecha de acceso: mayo 5, 2026, <https://samza.apache.org/case-studies/>
47. Core concepts - Apache Samza, fecha de acceso: mayo 5, 2026, <https://samza.apache.org/learn/documentation/1.0.0/core-concepts/core-concepts>
48. A Performance Comparison of Open-Source Stream Processing, fecha de acceso: mayo 5, 2026, <https://www.gta.ufri.br/ftp/gta/TechReports/ALD16b.pdf>
49. Kafka vs Spark - a comparison of stream processing tools - Quix, fecha de acceso: mayo 5, 2026, <https://quix.io/blog/kafka-vs-spark-comparison>
50. Exactly-Once vs At-Least-Once: Choosing Delivery Guarantees, fecha de acceso: mayo 5, 2026, <https://streamkap.com/resources-and-guides/exactly-once-vs-at-least-once>
51. Data Processing Guarantees Explained: Exactly-Once, At-Least, fecha de acceso: mayo 5, 2026, <https://chengzhizhao.com/data-processing-guarantees-explained-exactly-once-at-least-once/>

[east-once-and-at-most-once/](#)

52. How to understand Flink exactly-once and at-least-once semantics, fecha de acceso: mayo 5, 2026,
<https://stackoverflow.com/questions/78125414/how-to-understand-flink-exactly-once-and-at-least-once-semantics>
53. Understanding Stream Processing Guarantees in Apache Kafka, fecha de acceso: mayo 5, 2026,
https://medium.com/@_sidharth_m_/understanding-stream-processing-guarantees-in-apache-kafka-068eef52b3a7
54. Flink Vs. Kafka Streams: A Deep Dive - Ftp, fecha de acceso: mayo 5, 2026,
<https://ftp.bills.com.au/lunar-tips/flink-vs-kafka-streams-a-deep-dive-1764797462>